BHARATIYA VIDYA BHAVAN'S
SARDAR PATEL INSTITUTE OF TECHNOLOGY
Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai – 400058-India
Department of Computer Engineering

Name:	Nitin Gaikwad
UID no:	2025301008
Division & Batch:	SE - CE (Division - D & Batch – D)
Experiment No:	05

Aim:	To implement the Ford-Fulkerson algorithm for finding the maximum flow in a flow network and the Bipartite Matching algorithm using Depth First Search (DFS).
Objective:	1. Understand the concept of flow networks and maximum flow. 2. Learn the Ford-Fulkerson algorithm and its application. 3. Understand Bipartite Graphs and the maximum bipartite matching problem. 4. Implement bipartite matching using augmenting paths and DFS
Theory:	**1. Ford-Fulkerson Algorithm** The **Ford-Fulkerson algorithm** is used to find the **Maximum Flow** in a network. Imagine a network of water pipes; each pipe has a limit (capacity). This algorithm helps us find the maximum amount of water that can flow from the start (**Source**) to the end (**Sink**). **Key Concepts** ● **Flow Network:** A directed graph where every edge has a **capacity** (the maximum limit it can carry). ● **Source (S):** The node where the flow starts. ● **Sink (T):** The node where the flow ends. ● **Residual Graph:** A "leftover" graph that shows how much more flow can be sent through each edge. If a pipe can hold 10 units and is currently holding 4, the residual capacity is 6. ● **Augmenting Path:** A simple path from the Source to the Sink in the residual graph where more flow can still pass. ● **Bottleneck Capacity:** The smallest capacity along an augmenting path. This

value determines how much extra flow we can add to that entire path.

How the Algorithm Works

1. Start with a flow of 0 on all edges.
2. Look for an **augmenting path** from Source to Sink (using DFS or BFS).
3. Find the **Bottleneck Capacity** (the smallest edge) on this path.
4. Add this bottleneck value to the total flow.
5. Update the **Residual Graph** by subtracting the flow from the edges.
6. Repeat these steps until no more paths can be found.

---

2. Bipartite Graphs and Matching

A **Bipartite Graph** is a special graph where vertices are divided into two separate sets, let's call them **U** and **V**.

- Edges only go from a node in **U** to a node in **V**.
- There are **no edges** between nodes in the same set (no U→U or V→V).

Maximum Bipartite Matching

Matching means pairing one element from set **U** with one element from set **V**. For example, matching "Students" to "Internships."

- **Maximum Matching:** Finding the highest number of pairs possible so that no student has two jobs and no job is given to two students.

Solving Bipartite Matching using DFS

We can solve this by trying to find a "match" for every node in the first set:

1. Pick a node from the first set (e.g., an Applicant).
2. Use **DFS** to see if there is an available node in the second set (e.g., a Job).
3. If the job is free, match them.
4. If the job is already taken, check if the person who currently has that job can be moved to a different free job (**Augmenting Path**).
5. If you can move them, update the matches and increase the total count.

Transformation into a Flow Network

You can also solve Bipartite Matching using the **Ford-Fulkerson** method:

1. Create a "Super Source" and connect it to all nodes in set **U**.
2. Create a "Super Sink" and connect all nodes in set **V** to it.
3. Give every edge a capacity of **1**.
4. The Maximum Flow calculated will be equal to the **Maximum Matching**.

Code :

1. Ford-Fulkerson Algorithm (Maximum Flow)

This code finds the maximum flow from node 0 (Source) to node 5 (Sink) using BFS to find augmenting paths.

```
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>

#define V 6 // Number of vertices

// Function to find a path from source to sink in residual graph
bool bfs(int rGraph[V][V], int s, int t, int parent[]) {
   bool visited[V] = {false};
   int queue[V], front = 0, rear = 0;

   queue[rear++] = s;
   visited[s] = true;
   parent[s] = -1;

   while (front < rear) {
       int u = queue[front++];
       for (int v = 0; v < V; v++) {
           if (!visited[v] && rGraph[u][v] > 0) {
               if (v == t) {
                   parent[v] = u;
                   return true;
               }
               queue[rear++] = v;
               parent[v] = u;
               visited[v] = true;
           }
       }
```

```
    }
    return false;
}

int fordFulkerson(int graph[V][V], int s, int t) {
    int u, v;
    int rGraph[V][V]; // Residual graph
    for (u = 0; u < V; u++)
        for (v = 0; v < V; v++)
            rGraph[u][v] = graph[u][v];

    int parent[V];
    int max_flow = 0;

    // While there is a path from source to sink
    while (bfs(rGraph, s, t, parent)) {
        int path_flow = INT_MAX;

        // Find bottleneck capacity
        for (v = t; v != s; v = parent[v]) {
            u = parent[v];
                if (rGraph[u][v] < path_flow) path_flow =
rGraph[u][v];
        }

        // Update residual capacities
        for (v = t; v != s; v = parent[v]) {
            u = parent[v];
            rGraph[u][v] -= path_flow;
            rGraph[v][u] += path_flow;
        }
        max_flow += path_flow;
    }
    return max_flow;
}

int main() {
    int graph[V][V] = {
        {0, 10, 10, 0, 0, 0},
        {0, 0, 2, 4, 8, 0},
        {0, 0, 0, 0, 9, 10},
        {0, 0, 0, 0, 0, 10},
        {0, 0, 0, 6, 0, 10},
        {0, 0, 0, 0, 0, 0}
```

```
    };
        printf("The maximum possible flow is %d\n",
fordFulkerson(graph, 0, 5));
    return 0;
}
```

2. Maximum Bipartite Matching (DFS)

This code matches **Applicants** to **Jobs**. It uses DFS to see if an applicant can be assigned a job or if an existing assignment can be changed to make room.

```
#include <stdio.h>
#include <stdbool.h>
#include <string.h>

#define M 5 // Number of Applicants
#define N 4 // Number of Jobs

int bpGraph[M][N]; // Relationship matrix
int matchR[N];     // Stores which applicant is assigned
to which job
bool seen[N];      // To keep track of jobs already
visited in DFS

bool bpm(int u, int n) {
    for (int v = 0; v < n; v++) {
        // If applicant u is interested in job v and job v
is not visited
        if (bpGraph[u][v] && !seen[v]) {
            seen[v] = true;

            // If job v is not assigned OR the previous
applicant can get another job
            if (matchR[v] < 0 || bpm(matchR[v], n)) {
                matchR[v] = u;
                return true;
            }
        }
    }
    return false;
}

int maxBPM(int m, int n) {
```

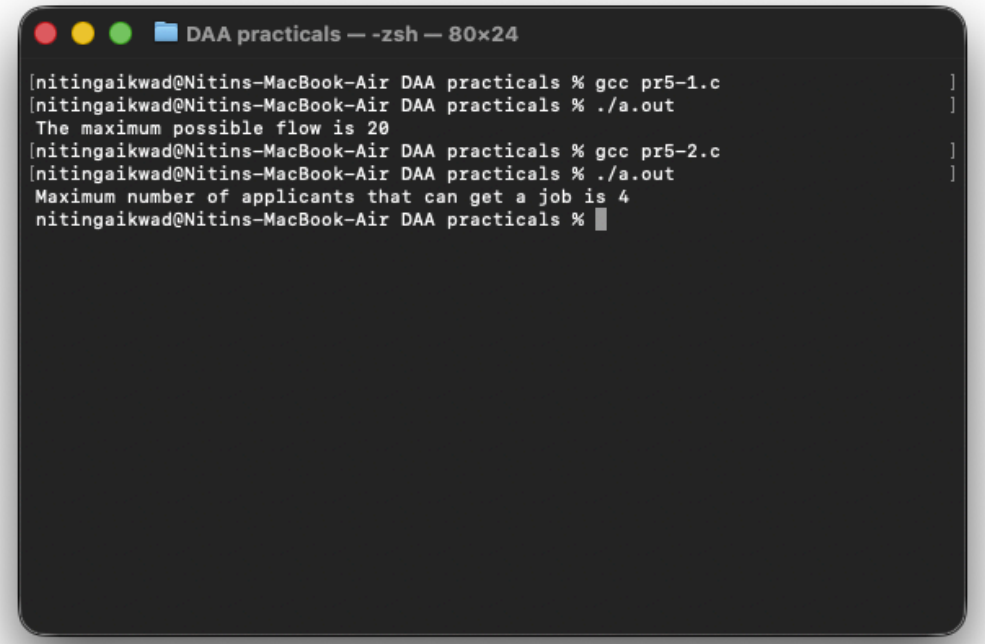
```
   memset(matchR, -1, sizeof(matchR)); // Initialize all
jobs as free
   int result = 0;
   for (int u = 0; u < m; u++) {
       memset(seen, false, sizeof(seen)); // Reset seen
for every applicant
       if (bpm(u, n))
           result++;
   }
   return result;
}

int main() {
   // 1 means applicant 'i' is interested in job 'j'
   int graph[M][N] = {
       {1, 0, 1, 0},
       {0, 1, 0, 1},
       {1, 1, 0, 0},
       {0, 0, 1, 1},
       {0, 1, 1, 0}
   };

   for(int i=0; i<M; i++)
       for(int j=0; j<N; j++)
           bpGraph[i][j] = graph[i][j];

   printf("Maximum number of applicants that can get a job
is %d\n", maxBPM(M, N));
   return 0;
}
```

Output:


DAA practicals — -zsh — 80×24
nitingaikwad@Nitins-MacBook-Air DAA practicals % gcc pr5-1.c
nitingaikwad@Nitins-MacBook-Air DAA practicals % ./a.out
The maximum possible flow is 20
nitingaikwad@Nitins-MacBook-Air DAA practicals % gcc pr5-2.c
nitingaikwad@Nitins-MacBook-Air DAA practicals % ./a.out
Maximum number of applicants that can get a job is 4
nitingaikwad@Nitins-MacBook-Air DAA practicals %

Analysis:

1. Detailed Analysis

A. Ford-Fulkerson Algorithm

- **Time Complexity:** O(E×f), where E is the number of edges and f is the maximum flow. We use BFS/DFS to find paths, and each path adds at least 1 unit of flow.
- **Space Complexity:** O(V2) because we store the Residual Graph in a 2D matrix.
- **Key Logic:** It uses a "Greedy" approach. It finds any path from source to sink and adds flow. The **Reverse Edges** in the residual graph are important because they allow the algorithm to "undo" or change a previous flow choice.

B. Maximum Bipartite Matching (MBM)

- **Time Complexity:** O(V×E). We run a DFS for each applicant to find a job.
- **Space Complexity:** O(V+E) to store the graph and the matching array.
- **Key Logic:** It uses **Backtracking**. If Job 1 is already taken by Applicant A, but Applicant B also wants Job 1, the algorithm checks if Applicant A can move to a different free job.

---

2. Dry Run: Ford-Fulkerson

Source (S) = 0, Sink (T) = 5

- **Iteration 1:**
  - **Path:** 0→1→4→5
  - **Bottleneck:** min(10,8,10)=8
  - **Total Flow:** 8
- **Iteration 2:**
  - **Path:** 0→2→5
  - **Bottleneck:** min(10,10)=10
  - **Total Flow:** 8+10=18
- **Iteration 3:**
  - **Path:** 0→1→3→5
  - **Bottleneck:** min(2,4,10)=2 (Note: 0→1 capacity was reduced to 2 in Iteration 1)
  - **Total Flow:** 18+2=20
- **Iteration 4:**
  - **Result:** No more paths available from 0 to 5.

- **Final Max Flow:** 20

---

3. Dry Run: Bipartite Matching

Matching 5 Applicants (A0-A4) to 4 Jobs (J0-J3)

1. **A0:** Wants J0. J0 is free. **Match: (A0-J0)**.
2. **A1:** Wants J1. J1 is free. **Match: (A1-J1)**.
3. **A2:** Wants J0.
   - J0 is taken by A0.
   - Can A0 move? Yes, A0 also likes J2. J2 is free.
   - **Update:** A0 moves to J2. A2 takes J0. **Matches: (A0-J2, A1-J1, A2-J0)**.
4. **A3:** Wants J2.
   - J2 is taken by A0.
   - Can A0 move? A0 likes J0 and J2 (both taken).
   - Check A3's next choice: A3 likes J3. J3 is free.
   - **Match: (A3-J3)**.
5. **A4:** Wants J1.
   - J1 is taken by A1.
   - Can A1 move? A1 likes J3 (taken).
   - Can A3 move? A3 likes J2 (taken).
   - No more free jobs or paths.
6. **Final Result:** Max matching is **4**.

Conclusion:	In this experiment, I successfully implemented the Ford-Fulkerson and Maximum Bipartite Matching algorithms using C programming. I learned how the Ford-Fulkerson algorithm uses a residual graph to find the maximum possible flow from a source to a sink. The concept of an augmenting path helped me understand how flow can be increased until no more paths are available. In the bipartite matching part, I used Depth First Search to pair applicants with available jobs efficiently. I observed that backtracking is essential for reassigning matches to achieve the best possible global result. Both algorithms are very useful for solving real-world problems like traffic routing and resource allocation. Overall, this practical provided a clear understanding of flow networks and their applications in computer science.